

**Smart Waste Classification**

# CLASSIFICATION AUTOMATIQUE DES DÉCHETS PAR APPRENTISSAGE PROFOND

**Transfer Learning · MobileNetV2 · 6 catégories de déchets**

*Analyse et modélisation avec Machine Learning*

*Présenté par:*

**Ghada Ben Mansour**

**Groupe 3**



# PLAN



- 1 Presentation du projet
- 2 Les donnees
- 3 Modelisation
- 4 Structure du projet
- 5 Resolution du probleme : desequilibre
- 5 Resultats et comparaison
- 5 App Streamlit et conclusion



# 1. PRÉSENTATION DU PROJET

## Contexte

Le tri des déchets est un problème mondial : une mauvaise séparation contamine les filières de recyclage et augmente les coûts de traitement. L'automatisation de cette tâche par vision par ordinateur représente une solution concrète et scalable.

## Ce que fait ce projet

Mon projet consiste à classifier automatiquement des images de déchets en six catégories.

J'ai utilisé un CNN, car ce type de réseau est très efficace pour extraire les caractéristiques visuelles des images comme les formes et les textures.

J'ai choisi MobileNetV2 avec le Transfer Learning, c'est-à-dire un modèle déjà pré-entraîné sur des millions d'images puis adapté à mon dataset.

Ce choix m'a permis d'obtenir une précision de 89,6 % rapidement, même avec un petit dataset de seulement 2527 images. »

## Les 6 catégories ciblées

Carton · Verre · Métal · Papier · Plastique · Déchets divers (Trash)

## Défi particulier

La catégorie "Trash" n'avait que 137 images

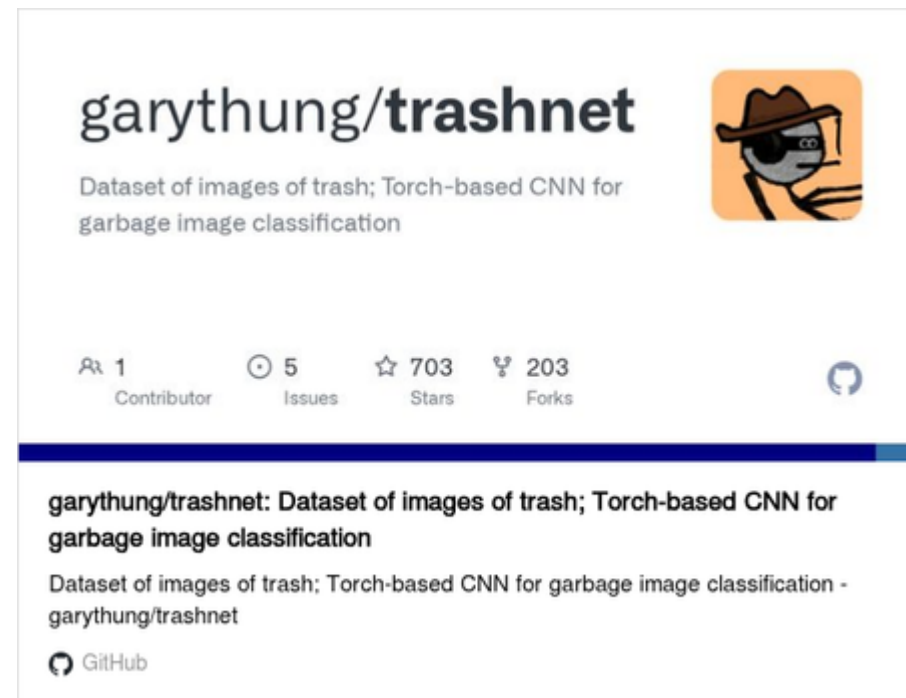
**soit 4 fois moins que les autres classes.**

Ce déséquilibre a causé de mauvaises performances sur cette classe, et le corriger est devenu l'axe principal d'amélioration du projet.

## 2. LES DONNÉES

### Chargement du dataset

Garbage Classification (Gary Thung GitHub)



### Taille

**Total initial : 2 527 images**

Taille totale du dataset : 54.56 MB

Taille moyenne par image : 0.0186 MB

### Répartition initiale des données :

```
cardboard : 402 images
glass : 501 images
metal : 409 images
paper : 594 images
plastic : 482 images
trash : 137 images
```

**Total initial : 2 527 images**

### Division des donnees

- 70 % entraînement
- 15 % validation
- 15 % test



## 2. LES DONNÉES

*Prétraitement appliqué :*

### Redimensionnement à 224 × 224 pixels

MobileNetV2 a été conçu pour recevoir des images de cette taille exacte. Une image de téléphone (4000×3000) est trop lourde et incompatible avec le réseau.

### Normalisation des pixels entre -1 et 1

- Cela centre les valeurs autour de 0, stabilise les calculs internes et correspond à ce que MobileNetV2 attend (il a été entraîné avec cette normalisation).





### 3. LE MODÈLE : TRANSFER LEARNING AVEC MOBILENETV2 (CNN)

#### Qu'est-ce que le Transfer Learning ?

Au lieu d'entraîner un modèle de zéro, on réutilise un modèle déjà entraîné sur des millions d'images

ImageNet

On garde ses connaissances visuelles et on lui apprend seulement à distinguer nos 6 catégories de déchets.

#### Pourquoi un CNN et pas un MLP ?

Un MLP (réseau classique) traite chaque pixel indépendamment : il ne comprend pas qu'un pixel est lié à ses voisins.

Un CNN utilise des filtres qui glissent sur l'image et détectent des patterns locaux

#### Pourquoi MobileNetV2 et pas un autre cnn ?

1. Depthwise separable convolutions : on découpe en deux étapes légères : une qui détecte les patterns par couleur, une qui les combine.
2. Inverted residual blocks avec skip connections : le bloc agrandit les features, les traite, puis les recomprime.

#### Fonctions de perte et optimiseurs :

##### Fonction de perte

`sparse_categorical_crossentropy` : adaptée aux labels entiers (0, 1, 2...)

##### Optimiseur

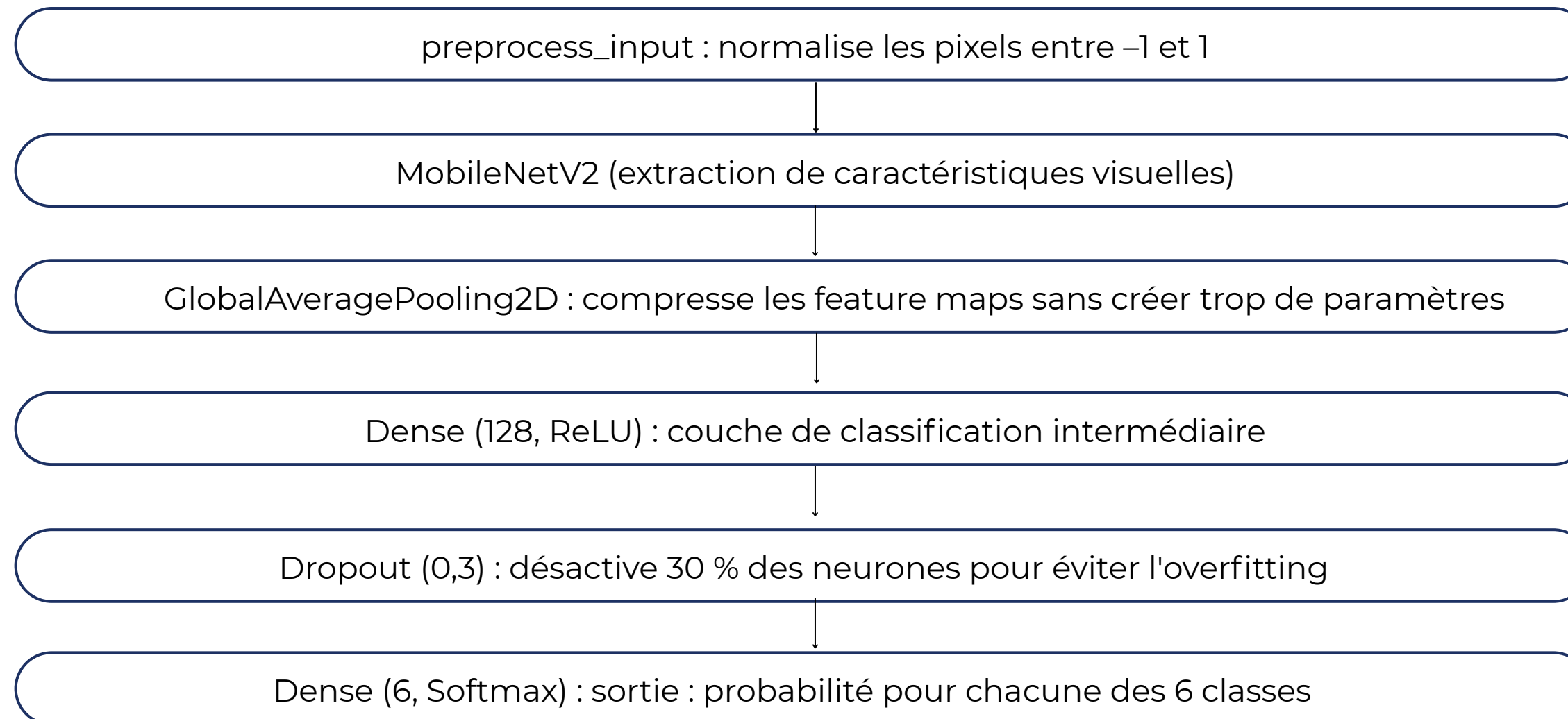
Adam



### 3. LE MODÈLE : TRANSFER LEARNING AVEC MOBILENETV2 (CNN)

*Architecture du modèle :*

*Entrée (photo 224x224)*



# 4. STRUCTURE DU PROJET

## Organisation des fichiers

Le projet est structuré en 3 grandes parties :

### 1. Les notebooks (progression documentée)

| Notebook                                      | Rôle                                    |
|-----------------------------------------------|-----------------------------------------|
| data_exploration.ipynb                        | Analyse et visualisation du dataset     |
| training_MobileNet_CORRECTED.ipynb            | Entraînement du modèle baseline         |
| evaluation_testing.ipynb                      | Évaluation du modèle baseline           |
| experiments_improvements.ipynb                | 3 expériences d'amélioration (V1)       |
| evaluation_exp3.ipynb                         | Évaluation du meilleur modèle           |
| data_augmentation_trash.ipynb                 | Augmentation offline de la classe Trash |
| experiments_improvements_after_data_aug.ipynb | Expériences après augmentation (V2)     |
| evaluation_after_data_aug.ipynb               | Évaluation du meilleur modèle           |

### 2. Les modèles sauvegardés

- 1.waste\_classifier.keras : modèle baseline
- 2.exp3\_combined.keras : Exp 3 (sans augmentation de Trash)
- 3.v2\_exp3\_combined.keras : Exp 3 V2 (meilleur modèle)

### 3. L'application Streamlit

app.py : interface de comparaison des deux modèles en temps réel



## 5. COMMENT ON A RÉSOLU LE DÉSÉQUILIBRE : DATA AUGMENTATION

### Le problème

La classe Trash avait 137 images : largement insuffisant. Le modèle "évitait" de prédire Trash car c'était rarement la bonne réponse.

### Résultat

**Accuracy globale : 84,5 %**  
**Mais recall de Trash : 70,4 %**

### Expérience 1 : Class Weights :

On pénalise davantage les erreurs sur Trash pendant l'entraînement. Le modèle y fait plus attention, sans toucher aux données.

**Résultat : recall Trash +15 pts : mais accuracy globale limitée**

### Expérience 2 : Fine-tuning

On "dégèle" les 30 dernières couches de MobileNetV2 pour les adapter à notre dataset, avec un très faible taux d'apprentissage ( $1e-5$ ).

**Résultat : meilleure accuracy globale : mais Trash toujours fragile**

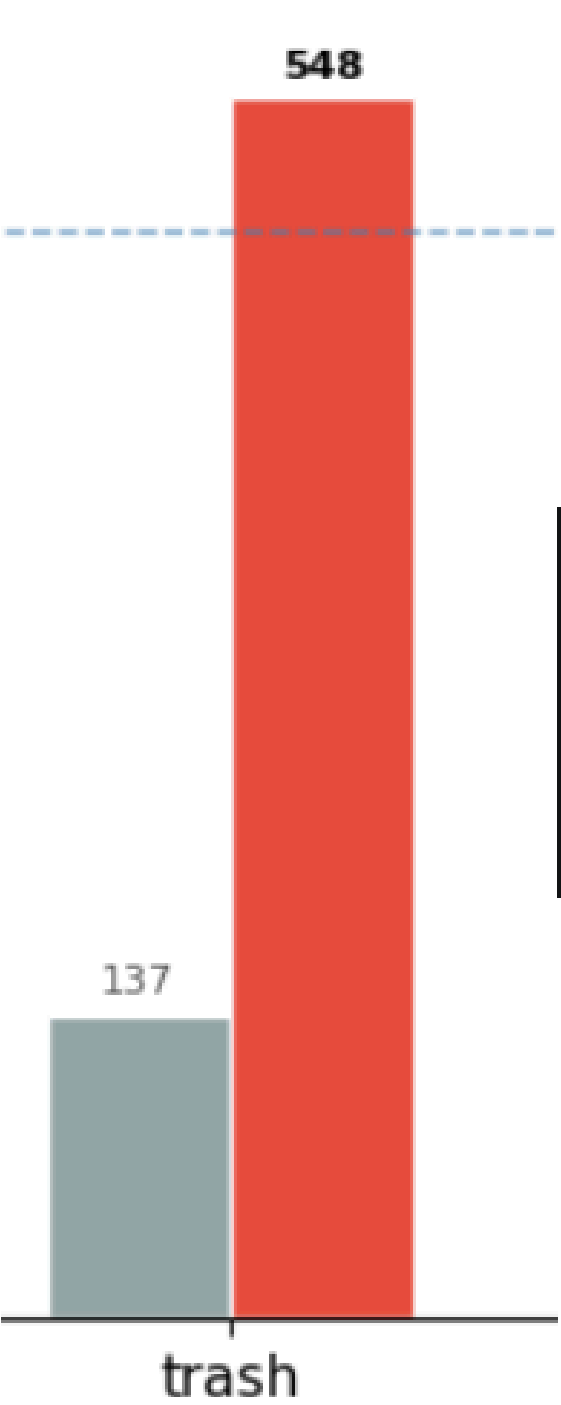
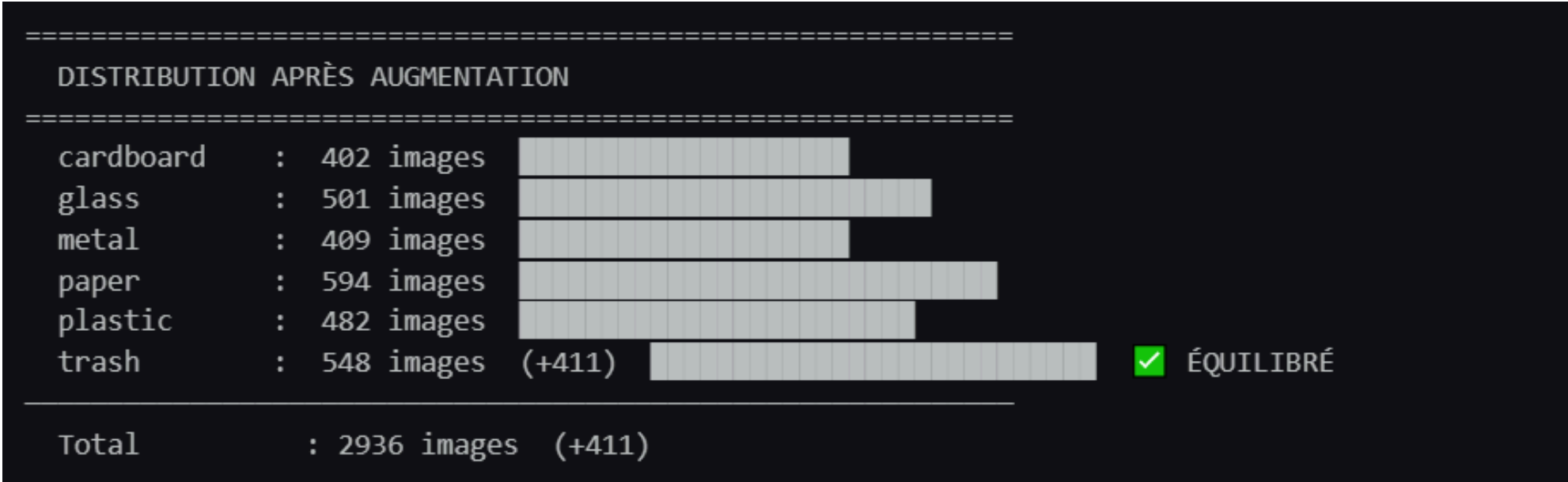
### Expérience 3 1/2 : Data Augmentation + Class Weights + Fine-tuning

On génère physiquement de nouvelles images Trash sur le disque (flip, rotation, zoom, luminosité). La classe passe de 137 à 548 images. On combine ensuite les class weights et le fine-tuning.

**Meilleur résultat**

**Pourquoi c'est la bonne approche ? Les class weights compensent le déséquilibre. L'augmentation le supprime vraiment.**

# 5. COMMENT ON A RÉSOLU LE DÉSÉQUILIBRE : DATA AUGMENTATION



cardboard : 7.23 MB  
glass : 6.37 MB  
metal : 6.93 MB  
paper : 12.25 MB  
plastic : 6.62 MB  
trash : 15.17 MB

# 6. RÉSULTATS ET COMPARAISON DES MODÈLES

Progression de l'accuracy : Recall Trash : 70,4 % → 88,0 %

| Version  | Technique              | Accuracy | Recall Trash |
|----------|------------------------|----------|--------------|
| Baseline | Transfer Learning seul | 84,5 %   | 70,4 %       |
| Exp 1    | + Class Weights        | ~86 %    | ~85 %        |
| Exp 2    | + Fine-tuning          | ~87 %    | ~82 %        |
| Exp 3 V2 | + Data Aug + CW + FT   | 89,6 %   | 88,0 %       |

Performance du meilleur modèle (Exp 3 V2) par classe

|                                          |           |        |          |         |
|------------------------------------------|-----------|--------|----------|---------|
| ÉVALUATION – V2 Exp 3 – CW + Fine-tuning |           |        |          |         |
| Test Accuracy : 89.62%                   |           |        |          |         |
| Test Loss : 0.3515                       |           |        |          |         |
| vs Baseline : 84.50% (+5.12%)            |           |        |          |         |
|                                          | precision | recall | f1-score | support |
| cardboard                                | 0.907     | 0.925  | 0.916    | 53      |
| glass                                    | 0.841     | 0.881  | 0.860    | 84      |
| metal                                    | 0.863     | 0.932  | 0.896    | 74      |
| paper                                    | 0.924     | 0.924  | 0.924    | 105     |
| plastic                                  | 0.828     | 0.828  | 0.828    | 64      |
| trash                                    | 1.000     | 0.880  | 0.936    | 92      |

Pourquoi plusieurs métriques ?

L'accuracy seule est trompeuse sur un dataset déséquilibré. On utilise 4 métriques complémentaires :

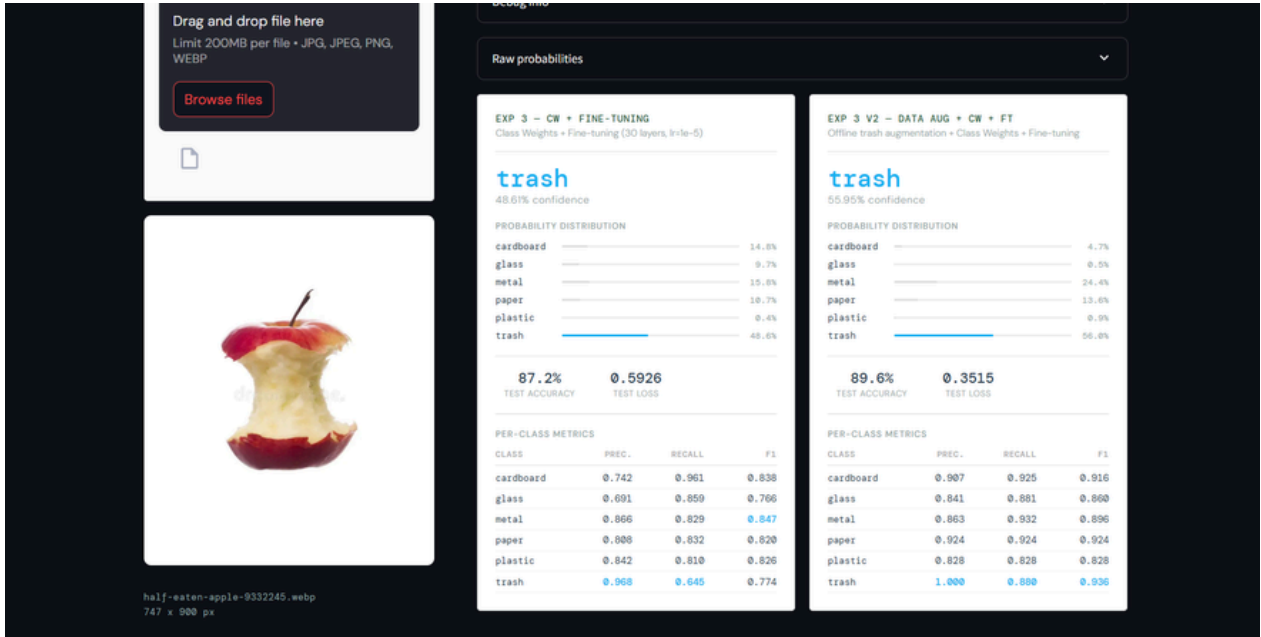
Accuracy : vue globale : 89,6 % des images correctement classées

Loss : qualité des probabilités : plus elle est basse, plus le modèle est confiant et correct

Precision : quand le modèle dit "Trash", a-t-il raison ? (Trash : 1,000 = zéro fausse alarme)

Recall : parmi tous les vrais Trash, combien le modèle en trouve-t-il ? (88,0 % en V2 vs 70,4 % en baseline)

F1-score : combine precision et recall en un seul chiffre par classe. Notre indicateur principal de comparaison entre expériences.

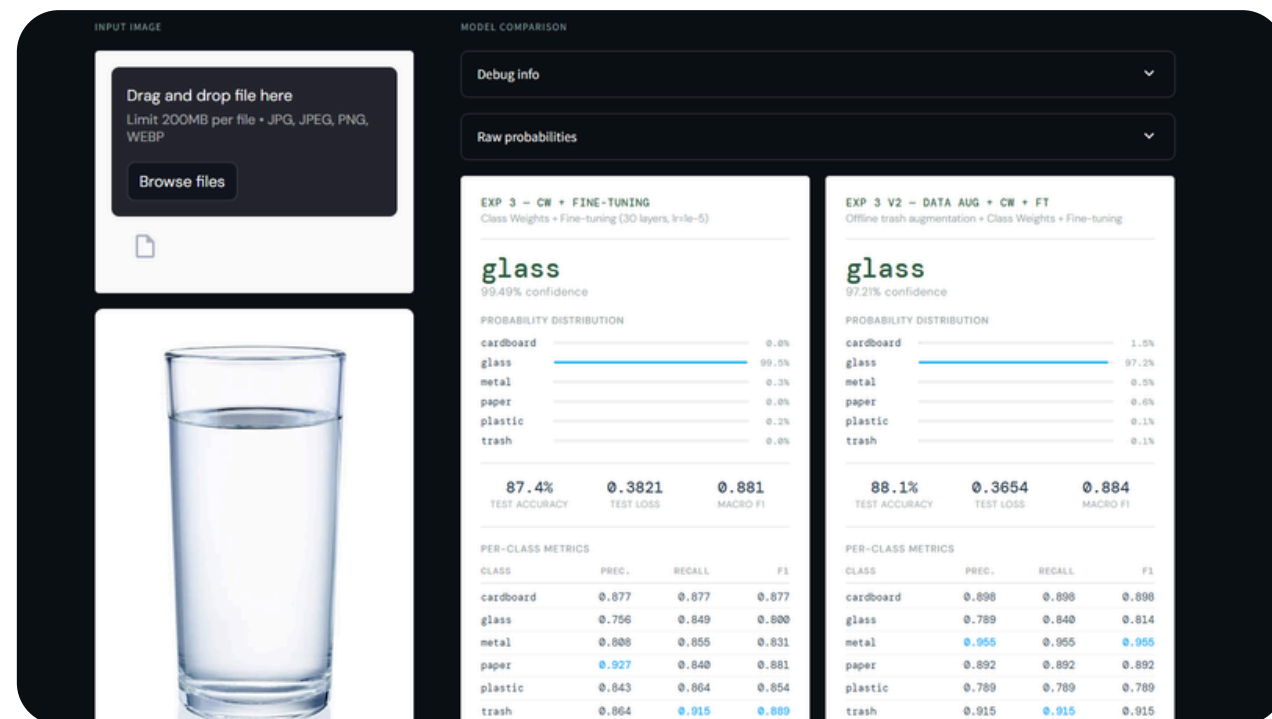


## 7. APPLICATION STREAMLIT & CONCLUSION

### L'application de démonstration

Une interface web développée avec Streamlit permet de :

- Uploader une photo de déchet
- Obtenir la prédiction des deux modèles en simultané (Exp 3 et Exp 3 V2)
- Voir la distribution des probabilités pour chaque classe
- Comparer les métriques des deux modèles côte à côte
- Visualiser si les deux modèles sont d'accord ou non sur la prédiction



### Ce qu'on retient de ce projet

L'augmentation offline est plus efficace que les class weights seuls quand les données manquent vraiment

Le Transfer Learning permet d'atteindre 89,6 % avec seulement 2 527 images

Les métriques par classe (recall, F1) sont indispensables : l'accuracy globale masquait le problème Trash

MobileNetV2 est plus adapté qu'EfficientNet sur un petit dataset : plus rapide, plus stable, meilleur résultat

Résultat final : De 84,5 % à 89,6 % de précision ---- Recall Trash de 70,4 % à 88,0 %



**MERCI DE VOTRE  
ATTENTION !**